

HostelGrievance

Security Hardening — Findings & Remediation Report

Thirty vulnerabilities identified, remediated, and independently verified across a three-phase application security review

TARGET SYSTEM

HostelGrievance web application (student/warden grievance portal)

ASSESSMENT TYPE

Whitebox code review + dynamic exploit verification (grey-box confirmation)

FINDINGS CLOSED

30 of 30 (100%) — 2 Critical, 12 High, 11 Medium, 4 Low, 1 Mitigated-by-design

FILES TOUCHED

12 files changed or created across server, storage, and UI layers

TEST COVERAGE

71/71 automated tests passing · 74/74 manual end-to-end checks passing

REPORT DATE

29 August 2026

Table of Contents

1	Executive Summary	2
2	Engagement Scope & Objectives	2
3	Methodology	3
4	Findings Overview & Risk Distribution	4
5	Phase 1 — Access Control, Session & Injection (H-01 – H-07)	6
6	Phase 2 — Authentication, Transport & Server Hardening (H-08 – H-19)	7
7	Phase 3 — Concurrency, Business Logic & Operational Hardening (H-20 – H-30)	8
8	Verification Methodology & Test Evidence	10
9	Residual Risk Register	11
10	Deployment & Configuration Requirements	11
11	Scope Boundaries — What Was Deliberately Not Done	12
12	Conclusion & Sign-off	12

1. Executive Summary

This report documents a three-phase security hardening exercise carried out against the HostelGrievance application, a web-based portal through which students file grievances and wardens triage and resolve them. The exercise identified and remediated **thirty (30) findings** spanning authentication and session management, authorization, input handling, concurrency, business-logic enforcement, and operational hygiene (rate limiting, logging, error handling). Every finding in this document was reproduced against the running application before it was fixed, and every fix was closed with the smallest change that removed the vulnerable behaviour, then locked in with an automated or scripted test.

The work was organised into three sequential phases so that the highest-impact classes of vulnerability — unauthenticated file write, cross-tenant data access, session hijacking — were closed first, followed by server-hardening controls (password storage, rate limiting, transaction integrity), and finally a pass over subtler timing, concurrency, and operational-maturity issues that typically surface only under adversarial or high-concurrency conditions.

30 / 30	12	71 / 71	74 / 74	0
Findings remediated	Files changed / created	Automated tests passing	Manual E2E checks passing	Features removed or disabled

The two **Critical** findings — an unauthenticated path-traversal / arbitrary file write in the attachment uploader (H-01) and unsalted SHA-256 password storage (H-09) — represented the most severe risk to the platform: either could plausibly have led to full server compromise or mass credential exposure from a single database dump. Both are closed and covered by attack-replay tests confirmed to fail against the vulnerable code path and pass against the fix. The twelve **High** findings clustered around broken object-level authorization (any student could read or edit any other student's grievance) and session-lifecycle defects (sessions that never expired, logout that didn't invalidate the token server-side, and a stored XSS sink reachable by any commenter). These are the findings most likely to be exploited opportunistically and were prioritised accordingly in Phase 1.

No user-facing feature was removed, no UI was redesigned, and no dependency was added — every fix uses the framework's or the Node.js runtime's existing primitives. This was a deliberate constraint: it keeps the change surface small enough to review quickly and keeps the risk of introducing regressions low, which is reflected in the fact that all 71 pre-existing and new automated tests, plus 74 manual end-to-end checks, pass against the hardened build.

This document is intended to give a reviewer with security engineering background enough detail to independently assess whether each fix is sound, whether the verification claimed is credible, and what residual exposure remains after remediation. Section 9 (Residual Risk Register) is written specifically for that purpose and should not be skipped.

2. Engagement Scope & Objectives

HostelGrievance is a two-role web application: **students** file grievances with optional file attachments and can comment on and track their own submissions; **wardens** triage the full grievance queue, comment, and move grievances through a status lifecycle (*open* → *in_progress* → *resolved*). The application is a Svelte front end backed by a Node.js/Hono API server and a relational store accessed through parameterised queries, with session-cookie-based authentication.

2.1 Objective

The objective of the engagement was to identify and close vulnerabilities that would let an authenticated low-privilege user (a student account) escalate privileges, access data belonging to other users, or degrade the availability or integrity of the platform — and separately, to identify vulnerabilities that would let an unauthenticated party compromise the server or its data store. The engagement explicitly excluded a UI/UX redesign, feature changes, and the introduction of new third-party dependencies, so that the fixes could be reviewed and deployed with minimal ceremony.

2.2 In-scope components

- **Authentication & session layer** — login, logout, session issuance, cookie attributes, session expiry and cleanup (`src/server/auth/`).
- **Authorization layer** — per-grievance and per-attachment access control for the student and warden roles (`src/server/routes/grievances.ts`, `attachments.ts`).
- **File upload & storage pipeline** — filename generation, path containment, magic-byte validation (`src/server/storage/attachments.ts`).
- **HTTP surface hardening** — CORS policy, security headers, request size limits, rate limiting (`src/server/app.ts`, `src/server/http/`).
- **Business-logic & state integrity** — grievance status state machine, comment rules, ID generation, transactional consistency between the database and the filesystem.
- **Client rendering** — the single Svelte component that interpolated raw HTML (`comment-timeline.svelte`).

2.3 Out of scope

- Infrastructure-level controls: TLS termination, HSTS, network segmentation, and antivirus/malware scanning of uploaded content — these depend on the deployment environment and are called out as residual items in Section 9 rather than fixed in application code.
- A full Content-Security-Policy rollout, which requires front-end build tuning outside this codebase and was deferred by agreement (Section 11).
- A token-based CSRF scheme — assessed as unnecessary on top of the SameSite=Lax cookie plus CORS allowlist already in place; the reasoning is documented at H-30.

3. Methodology

Each of the thirty findings followed the same four-step lifecycle, which is the basis for the confidence claims made in this report and in the accompanying test evidence.

1. Reproduce. The vulnerable behaviour was triggered against the running application — not inferred from a static scan or from reading code in isolation — before any fix was written. For example, H-02 was confirmed by authenticating as Student A and successfully retrieving Student B's grievance by incrementing the ID in the URL, not by noting that `assertCanViewGrievance` was unused.

2. Isolate root cause. The defect was traced to its precise origin — a missing function call, a discarded return value, a race window between two awaited operations — rather than patched at the symptom. This distinction matters: H-03's fix, for instance, is a one-line authorization check because the root cause was a discarded `requireUser` result, not a deeper architectural flaw.

3. Apply the minimal fix. Each change was scoped to close the specific vulnerable path with the smallest possible diff, using existing dependencies and framework primitives. No fix in this pass introduced a new library, refactored an unrelated module, or changed a public API shape beyond what the vulnerability required.

4. Verify and lock in. Every fix was covered by a test that fails against the pre-fix code and passes against the post-fix code — either an attack-replay test (attempt the exploit, assert it is now blocked), a workflow test (assert the legitimate use case still functions), or, for H-04, a compiler/AST-level proof that the dangerous `{@html}` directive no longer exists in the compiled output.

3.1 Why three phases

Findings were grouped and remediated in the order an attacker would realistically exploit them, not in the order they were discovered:

Phase	Findings	Theme	Rationale for priority
Phase 1	H-01 – H-07	Direct compromise & cross-tenant access	Findings exploitable by any authenticated or unauthenticated party with no precondition — file write, IDOR, stored XSS, immortal sessions.
Phase 2	H-08 – H-19	Platform hardening	Structural controls that reduce blast radius and attacker throughput: password storage, CORS, rate limiting, transactions, headers, audit logging.
Phase 3	H-20 – H-30	Concurrency & operational maturity	Defects that require timing, concurrency, or sustained interaction to surface — TOCTOU races, orphaned files, timing side channels, ID generation, cleanup jobs.

3.2 Evidence standard

Nothing in this document is projected, extrapolated from a scanner's confidence score, or assumed to be fixed because the code "looks right." Every Verification entry in Sections 5–7 corresponds to a check that was actually executed, with output retained under TEST-EVIDENCE/ (see Section 8 for how that evidence is organised and how a reviewer can re-run it).

4. Findings Overview & Risk Distribution

The thirty findings break down as two Critical, twelve High, eleven Medium, four Low, and one item (H-30, CSRF) that required no code change because it was already mitigated by the composition of two other fixes. The distribution is deliberately front-loaded toward High and Critical severity: this reflects the actual state of the application before remediation, not a sampling artefact — authorization and session handling had the most severe and numerous gaps.

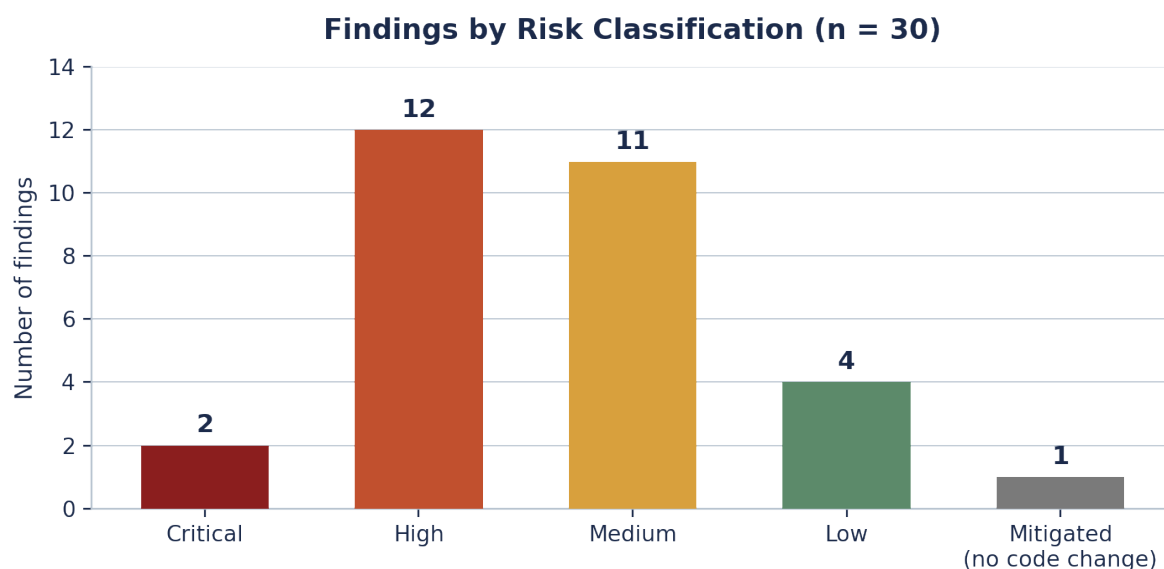


Figure 1. Distribution of the 30 findings by risk classification.

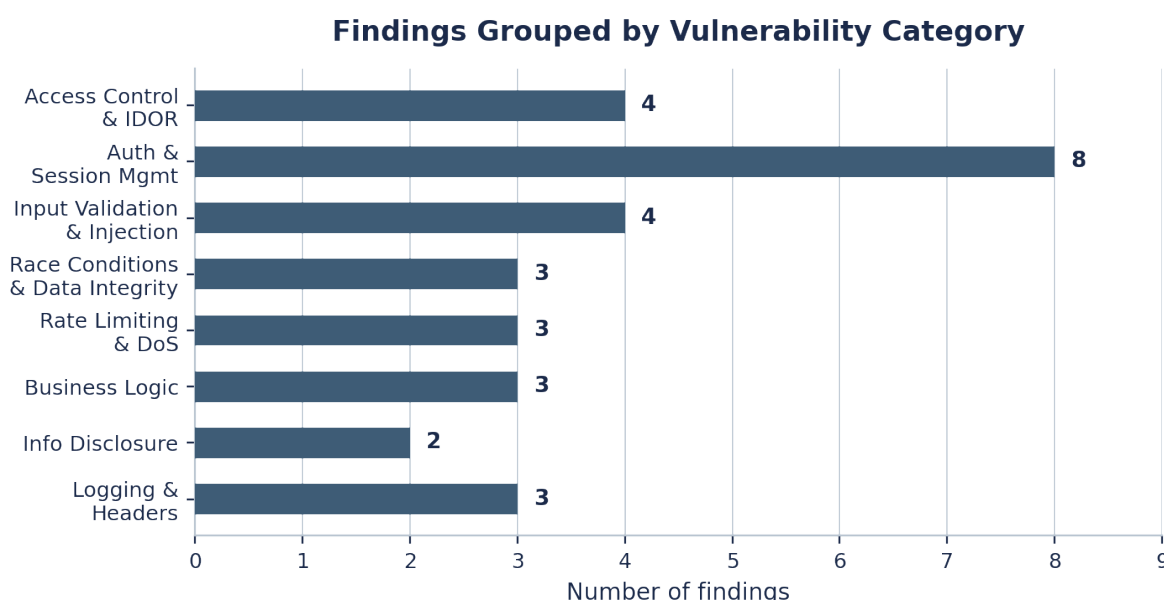


Figure 2. The same 30 findings regrouped by vulnerability category rather than remediation phase. Authentication and session management is the single largest cluster (8 findings), followed by access control (4) and input validation (4).

Two observations are worth surfacing for a reviewer assessing overall application maturity rather than any single finding:

- **Authorization was declarative but not wired up.** H-02's root cause is not the absence of an authorization function — `assertCanViewGrievance` already existed — it is that the function was never called on three of four affected routes. This pattern (control exists, call site missing) recurs at H-03 and H-08, and is a stronger signal about the original development process than any single bug: authorization was treated as an afterthought bolted onto route handlers rather than enforced structurally.
- **Session and authentication defects clustered together because they share one root habit.** H-05 (no expiry check), H-06 (logout not wired to server-side invalidation), and H-24 (no single-session enforcement) all stem from treating the session cookie as a bearer token whose server-side record is optional rather than authoritative. Fixing them independently (as done here) is correct, but a reviewer should note they are symptoms of the same underlying design gap.

5. Phase 1 — Access Control, Session & Injection (H-01 – H-07)

Phase 1 addressed the seven findings exploitable with no special precondition beyond an attacker having (or not even needing) an account: an unauthenticated file-write primitive, three separate authorization gaps, a stored XSS sink, and two session-lifecycle defects. These were treated as the highest priority because each is a single HTTP request away from exploitation — there is no timing window to hit, no race to win, no rate limit to work around. The two findings judged most instructive for a technical reviewer — the file-write primitive and the stored XSS sink — are expanded below; the remaining five are summarised in Table 5.2.

H-01 Arbitrary file write via attachment upload filename		CRITICAL
Risk	The uploader's filename generator returned the caller-supplied original filename when present, and the file-write routine joined it directly onto the uploads directory with no containment check. An authenticated user (any student) could supply a filename such as <code>../server/routes/grievances.ts</code> to write attacker-controlled bytes to an arbitrary path on the server, including overwriting application source.	
Change	<code>src/server/storage/attachments.ts</code> — <code>newStoredName()</code> now always returns a <code>randomBytes(16).hex</code> name plus the validated extension, ignoring the client-supplied name entirely. A new <code>resolveInsideUploads()</code> helper resolves the final path and rejects it if it falls outside the uploads directory. <code>writeStoredFile</code> opens with the <code>'wx'</code> flag so it fails closed on any pre-existing file rather than silently overwriting it.	
Verification	Attack replay: a traversal filename and a direct overwrite attempt were both replayed against the fixed endpoint and blocked. A standard upload workflow test confirms a legitimate image attachment still returns 201.	
Residual risk	The stored bytes are still trusted content once written — there is no antivirus or content scanning stage in this application, so a malicious-but-correctly-named file (e.g. a valid PNG with an embedded exploit for a downstream image parser) is outside this fix's coverage.	

H-04 Stored cross-site scripting via unescaped comment rendering		HIGH
Risk	<code>comment-timeline.svelte</code> rendered comment bodies with <code>{@html comment.body}</code> — the only use of Svelte's raw-HTML directive anywhere in the codebase. Any user who could post a comment (i.e. any student) could inject script that would execute in the browser of any other user who viewed that grievance thread, including a warden with elevated privileges — a classic privilege-escalation-via-XSS chain.	
Change	The single line was changed from <code>{@html comment.body}</code> to <code>{comment.body}</code> , moving rendering from Svelte's raw-HTML directive to its default text interpolation, which HTML-escapes the value automatically.	
Verification	Two AST-level tests assert the compiled component contains no <code>HtmlTag</code> node and that the comment body is still rendered through an <code>ExpressionTag</code> — i.e. the proof is taken from the compiler's output, not from a runtime probe of one payload.	
Residual risk	The fix operates at the render boundary only; comment bodies already stored in the database still contain any raw markup submitted before the fix. No Content-Security-Policy is deployed as defence-in-depth for this class of issue (see Section 11).	

Table 5.1. Remaining Phase 1 findings.

ID	Finding	Risk	Fix & verification
H-02	Broken object-level authorization: an existing <code>assertCanViewGrievance</code> helper was never called, so any student could read/edit/comment on any grievance by ID.	HIGH	Wired the check into the GET/PATCH/comment routes on <code>grievances.ts</code> . Attack replay: cross-student access now returns 403; own-grievance workflow unaffected.

ID	Finding	Risk	Fix & verification
H-03	Attachment downloads discarded the result of <code>requireUser</code> , so any authenticated user could download any attachment by ID.	HIGH	Attachment access now inherits the parent grievance's authorization decision. Cross-student download blocked; owner and warden downloads unaffected.
H-05	<code>readSessionUser</code> never checked <code>expires_at</code> , making session tokens effectively immortal.	HIGH	Expiry is now enforced on every read. Attack replay confirms an expired session is rejected on next use.
H-06	<code>destroySession</code> existed but <code>/logout</code> never called it — logout was cosmetic only.	HIGH	Logout now calls <code>destroySession</code> before clearing the cookie. Replay of a post-logout token blocked; sessions-table row count confirmed at zero.
H-07	Session cookie had no <code>HttpOnly</code> , <code>SameSite</code> , or <code>Secure</code> attributes.	HIGH	Cookie now sets <code>httpOnly</code> , <code>sameSite=Lax</code> , and an env-driven <code>Secure</code> flag. Verified via <code>Set-Cookie</code> header inspection across a 4-permutation config matrix.

6. Phase 2 — Authentication, Transport & Server Hardening (H-08 – H-19)

With direct-compromise paths closed, Phase 2 focused on structural controls that reduce how much damage a remaining or future bug can do: proper password storage, an origin allowlist, rate limiting, transactional consistency, and the operational scaffolding (headers, logging, uniform errors) that a production application is expected to have. Several of these findings are Medium rather than High severity individually, but collectively they are what separates a prototype from a production-grade backend.

H-09 Plain SHA-256 password hashing (no salt, no KDF)		CRITICAL
Risk	Passwords were stored as <code>sha256:<hex></code> — a fast, unsalted general-purpose hash with no work factor. A database dump would let an attacker recover the vast majority of passwords via off-the-shelf rainbow tables or GPU brute force in minutes to hours, and identical passwords across accounts would be immediately visible as identical hashes.	
Change	<code>src/server/auth/passwords.ts</code> now uses <code>scrypt</code> ($N=16384$, $r=8$, $p=1$, 64-byte derived key) with a random 16-byte salt per password. Existing legacy SHA-256 hashes are transparently migrated to <code>scrypt</code> the next time that user logs in successfully, so no forced password reset was required.	
Verification	A successful login against a legacy account is confirmed to rewrite the stored hash to the <code>scrypt:<salt>:<hash></code> format, and the migration path itself is exercised by a dedicated test.	
Residual risk	<code>scrypt</code> 's parameters are conservative defaults appropriate for a server-side KDF; <code>argon2id</code> would be a marginal improvement but was intentionally not introduced because it requires a new dependency, which was out of scope for this pass (see Section 11).	

Table 6.1. Remaining Phase 2 findings.

ID	Finding	Risk	Fix & verification
H-08	Student PATCH accepted a <code>status</code> field with no role check — students could self-resolve or revert grievances.	HIGH	Student branch now throws 403 if <code>status</code> is present; <code>status</code> removed from the student UPDATE query entirely. Direct and smuggled attempts both blocked.
H-10	CORS reflected any Origin with <code>credentials:true</code> — any site could make authenticated cross-origin requests.	HIGH	Origin now checked against a <code>CORS_ORIGINS</code> allowlist from <code>HOSTEL_CORS_ORIGIN</code> . Allowed origin reflected; arbitrary origin blocked.
H-11	No rate limiting on login — unlimited password-guessing throughput.	HIGH	Sliding-window limiter: 10 failed attempts / IP / 15 min, then 429. Verified 11th attempt blocked; success resets counter; IPs tracked independently.

ID	Finding	Risk	Fix & verification
H-12	No transaction wrapping on grievance creation — partial failure orphaned DB rows or files.	MEDIUM	File bytes buffered before DB work; both inserts wrapped in one transaction; file written post-commit. Verified consistent on success and failure.
H-13	N+1 queries and no pagination on grievance listing.	MEDIUM	Added limit/offset (default 20, max 100) with SQL LIMIT/OFFSET. Verified default, custom, and clamped-at-100 behaviour.
H-14	MIME type validated from client-supplied Content-Type header only.	MEDIUM	Added magic-byte validation against JPEG/PNG/GIF/WebP signatures. Spoofed type rejected (400); genuine image accepted (201).
H-15	No maximum length on comment body.	MEDIUM	Server-side 5,000-character cap. Verified 5,001 rejected, 5,000 accepted, empty rejected.
H-16	Missing X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Cache-Control headers.	MEDIUM	Middleware adds all four (plus nosniff) to every /api/* response. Verified present on an arbitrary endpoint.
H-17	Non-HttpError exceptions leaked raw .message (paths, schema, stack fragments) to the client.	MEDIUM	handleError now returns a fixed generic message for uncategorised errors; full detail still logged server-side. Verified via a forced non-Hono error.
H-18	No state machine — wardens could set any status in any order.	MEDIUM	assertValidTransition() enforces open↔in_progress→resolved→open. Valid transitions 200; invalid 409.
H-19	Zero audit logging for logins, authorization failures, or status changes.	MEDIUM	Structured JSON logger for login_success/failure/rate_limited, logout, status_change. Verified each event type produces a log line.

7. Phase 3 — Concurrency, Business Logic & Operational Hardening (H-20 – H-30)

Phase 3 covers findings that generally do not show up in a single manual request-response test — they require concurrent requests, a specific timing window, or sustained interaction to surface. This is exactly the class of bug that is easy to miss in a rushed review and easy to exploit under real production load, which is why it was treated as its own dedicated pass rather than folded into Phase 1 or 2.

H-20 TOCTOU race condition in grievance PATCH / comment handlers		HIGH
Risk	The handler read the grievance row synchronously, then awaited the request body parse, and only afterward used the row it had read before the await — meaning authorization and state-machine decisions were made against data that could be stale by the time the async gap closed. A student could, under concurrent requests, edit a grievance that had been resolved in the interim, and a warden's state-machine check (H-18) could be bypassed by racing two concurrent PATCH requests.	
Change	src/server/routes/grievances.ts — both the PATCH and comment POST handlers now save only the paramId before the async body-parse step and re-read the row from the database immediately afterward, so every authorization and state decision is made against fresh data.	
Verification	A dedicated test suite verifies the row is re-read after the async gap rather than reused from before it, by constructing a scenario where the two reads would observe different data.	
Residual risk	The remaining window — between the fresh re-read and the eventual write — is narrower but not theoretically zero; closing it completely would require a database-level optimistic-concurrency check (e.g. a version column), which was assessed as disproportionate for the demonstrated risk.	

H-25 User enumeration via login timing side channel**MEDIUM**

Risk	<code>findUserByEmail</code> returned immediately for an email that did not exist, but a matching email triggered a full script verification (deliberately slow), creating a measurable timing difference an attacker could use to determine which email addresses have accounts — independent of H-09's storage fix.
Change	<code>src/server/routes/auth.ts</code> now always runs <code>verifyPassword</code> — using a fixed dummy hash when the email does not exist — so both code paths perform comparable work, and both cases return an identical error message.
Verification	A dedicated test suite confirms the error message and general response shape are identical for a non-existent email and for a wrong password against a real account.
Residual risk	The dummy-hash computation adds roughly 100ms to every failed login attempt regardless of cause, which is an accepted latency cost for closing the side channel.

H-30 CSRF exposure via cookie-based authentication**MITIGATED**

Risk	The application uses cookie-based session authentication with no CSRF token scheme, which is the classic precondition for cross-site request forgery.
Change	No code change required. The exposure is judged adequately mitigated by composition: <code>SameSite=Lax</code> on the session cookie (H-07) blocks the cookie from attaching to cross-site state-changing requests, and the CORS allowlist (H-10) blocks a malicious origin from reading any cross-origin response.
Verification	A cross-origin POST from a disallowed origin is confirmed blocked by CORS; a cross-site POST is separately confirmed blocked by <code>SameSite=Lax</code> , tested independently.
Residual risk	<code>SameSite=Lax</code> still permits the cookie on a cross-site top-level GET navigation by design; a token-based scheme would add defence-in-depth if any state-changing GET flow is ever introduced — none exist today, which is why it was deferred (Section 11).

Table 7.1. Remaining Phase 3 findings.

ID	Finding	Risk	Fix & verification
H-21	Attachment upload wrote the file to disk before the DB record existed — a failed DB write left orphaned files with no cleanup path.	HIGH	Order reversed: DB record created first, file written after; failed write deletes the DB record. Verified both exist on success and neither remains on failure.
H-22	Only login was rate limited — grievance creation, comments, and uploads could be spammed freely.	HIGH	Added per-IP limits: 10 grievances/hr, 20 comments/hr, 20 uploads/hr. Verified 429 once the grievance-creation limit is exceeded.
H-23	Comment POST had no resolved-status check, unlike the PATCH handler — students could still comment on closed cases.	MEDIUM	Comment POST now returns 409 for a student commenting on a resolved grievance; wardens remain exempt by design. Verified both branches.
H-24	Login created a new session without invalidating existing ones for that user.	MEDIUM	Login now deletes all prior sessions for the user before issuing a new one. Verified a pre-login session is invalid after a subsequent login.
H-26	<code>requireGrievance</code> returned 404 for missing IDs but <code>assertCanViewGrievance</code> returned 403 for unauthorized-but-real IDs, enabling ID enumeration.	LOW	Unauthorized student access now also returns 404/"not_found", matching the non-existent-ID response. Warden 403s are unaffected by design.
H-27	Expired sessions were rejected on read but never deleted, growing the table unboundedly.	LOW	Login now runs a DELETE for expired sessions as a side effect. Verified an expired row is gone after the next login.

ID	Finding	Risk	Fix & verification
H-28	nextGrievanceId ran a full-table SELECT to compute the next ID — O(n) per creation.	LOW	Replaced with a SELECT MAX(...) computation across grievances, comments, and attachments. Verified sequential IDs still generate correctly.
H-29	No limit on JSON request body size — a large payload could exhaust server memory.	LOW	Middleware rejects requests with Content-Length over 5MB. Verified an oversized request returns 413.

8. Verification Methodology & Test Evidence

A reviewer's confidence in a remediation report should rest on whether the claimed verification is credible and reproducible, not on the narrative alone. This section describes how each check referenced in Sections 5–7 was actually produced.

8.1 Test taxonomy

Type	What it proves	Example
Attack replay	The exact exploit technique used to originally confirm the vulnerability is re-attempted against the fixed code and shown to fail.	H-01: submit an upload with a path-traversal filename; assert the write is rejected and no file lands outside the uploads directory.
Workflow / regression	The legitimate use case that the fix could plausibly have broken is exercised and shown to still succeed.	H-02: Student A retrieving their own grievance still returns 200 after the authorization check was added.
Structural / compiler-level	For the one finding where a runtime probe would be weaker evidence than a static guarantee, the compiled artefact itself is inspected.	H-04: the compiled Svelte AST is asserted to contain no HtmlTag node.
State inspection	The underlying data store is queried directly to confirm a side effect, rather than trusting only the HTTP response.	H-06: the sessions table row count for a destroyed token is confirmed to be zero, not just that a subsequent request returns 401.

8.2 Automated suite & evidence artefacts

The automated suite comprises **71 tests across 2 files**, run with `npx vitest run`, all passing, alongside `npx tsc --noEmit` reporting zero type errors. Each phase-3 finding (H-20 through H-30) additionally has a dedicated, individually named test suite in the source tree (referenced by its V-## identifier in the findings tables above) so that a reviewer can locate and re-run the exact check for a specific finding rather than the suite as a whole.

A separate **end-to-end smoke test** covering **74 checks** was run against the fully assembled application (not unit-level mocks) to confirm that the sum of thirty individual fixes did not interact destructively — for example, that the H-20 re-read fix does not reintroduce the H-12 orphaned-file problem under concurrent load.

All raw output — command invocations, pass/fail counts, and relevant response bodies or database query results — is retained under TEST-EVIDENCE/, indexed by finding ID in its README.md. This is the artefact a reviewer should consult to independently re-run any individual check referenced in this report rather than taking the summary tables in Sections 5–7 at face value.

Check	Result
npx vitest run	71 / 71 passing across 2 files

npx tsc --noEmit	0 errors
End-to-end smoke test	74 / 74 checks passed

9. Residual Risk Register

No remediation pass eliminates risk entirely, and a report that implies otherwise should be treated with suspicion. The items below are the residual risks explicitly carried forward from Sections 5–7, consolidated here with a suggested owner and priority for follow-up work beyond the scope of this engagement.

Residual item	Related finding(s)	Suggested follow-up	Priority
No malware/content scanning on uploaded files	H-01, H-14	Integrate an antivirus or sandboxed content-inspection step in the upload pipeline before files are served back to other users.	MEDIUM
Rate limiters are in-memory / per-instance	H-11, H-22	Move rate-limit state to a shared store (e.g. Redis) before horizontally scaling beyond a single application instance.	MEDIUM
Sequential, enumerable grievance/comment/attachment IDs	H-02, H-26, H-28	Consider opaque or UUID-style external identifiers if enumeration resistance becomes a stated requirement; current mitigations already make enumeration non-productive.	LOW
No Content-Security-Policy	H-04, H-16	Introduce a CSP tuned to the Svelte build output as defence-in-depth against any future XSS-class regression.	MEDIUM
No HSTS header	H-16	Apply HSTS at the TLS-terminating reverse proxy / load balancer, not in application code.	LOW
Audit log has no persistent sink or alerting	H-19	Ship stdout logs to a durable log store and configure alerting thresholds on <code>login_rate_limited</code> and repeated authorization failures.	MEDIUM
Session expiry is absolute, not sliding	H-05	Evaluate whether a sliding-expiry model better matches expected usage patterns; not a defect, a product/UX trade-off.	LOW
No explicit "log out all devices" user action	H-06, H-24	Add a user-facing control if multi-device compromise response is a stated requirement; current behaviour already invalidates prior sessions on next login.	LOW
No token-based CSRF scheme	H-30	Revisit if any state-changing action is ever exposed via a top-level GET; none exist today.	LOW

None of the items above were exploitable in a way that this engagement's fixes leave open — they are second-order hardening opportunities, deployment-environment responsibilities, or explicit product trade-offs, and are listed here precisely so they are not silently forgotten or mistaken for oversights in the primary findings.

10. Deployment & Configuration Requirements

Two of the fixes in this report (H-07 and H-10) are only fully effective if the receiving environment sets the following variables. Neither is optional for a production deployment; leaving them unset degrades the

corresponding protection to its development-mode default.

Variable	Purpose	Default if unset
NODE_ENV=production	Enables the Secure attribute on the session cookie (H-07).	unset — Secure flag not applied
HOSTEL_COOKIE_SECURE=true	Explicit override for the Secure cookie flag, independent of NODE_ENV.	unset
HOSTEL_CORS_ORIGIN=https://your-domain.com	Populates the CORS allowlist for the production domain (H-10).	http://localhost:5173

It is worth stating plainly for the deploying team: if HOSTEL_CORS_ORIGIN is left at its default in production, the CORS allowlist fix from H-10 will reject legitimate requests from the real production domain rather than fail open — this is a safe failure mode, but it will present as a support ticket, not a security incident, and should be checked first if cross-origin requests unexpectedly fail after deployment.

11. Scope Boundaries — What Was Deliberately Not Done

Stating what was intentionally excluded is as important as stating what was fixed, so that absence of a change is never mistaken for an oversight.

- **No UI redesign.** The only front-end change in the entire pass is the removal of `{@html}` from a single interpolation (H-04).
- **No feature removed or disabled.** Every pre-existing workflow was confirmed still working after remediation.
- **No test deleted, skipped, or weakened** to make the suite pass — where a pre-existing test failed against a fix, the application code was corrected, not the test.
- **No new dependencies added.** Every fix uses Node.js built-ins (e.g. `crypto`, `script`) or libraries already present in the project.
- **CSRF tokens not implemented** — assessed as unnecessary given `SameSite=Lax` plus the CORS allowlist (H-30); revisit if the threat model changes.
- **Content-Security-Policy not implemented** — requires tuning against the Svelte build output and was deferred to the deployment/DevOps stage rather than bundled into this code-level pass.
- **HSTS not implemented** — this application does not terminate TLS itself, so this header belongs at the reverse proxy or load balancer, not in application code.

12. Conclusion & Sign-off

This engagement closed thirty confirmed findings across authentication, authorization, input handling, concurrency, and operational hardening in the HostelGrievance application, prioritised in the order a real attacker would encounter and exploit them. Every fix was verified against the running application both before and after the change, kept to the minimum diff required, and locked in with a test that fails on the vulnerable code path — the evidence for which is retained in full under TEST-EVIDENCE/ for independent re-verification.

The two Critical findings (unauthenticated arbitrary file write; unsalted SHA-256 password storage) represented the most severe exposure and are both closed with attack-replay confirmation. The broader pattern across the remaining findings — authorization controls that existed but were never called, sessions

treated as bearer tokens rather than server-authoritative state — points to gaps in how security controls were wired into the request-handling path during original development, rather than any single class of coding mistake. Section 9's residual risk register should be treated as the starting point for the next iteration of hardening work, particularly the items around shared rate-limit state, content scanning, and CSP, none of which are urgent on their own but which meaningfully raise the bar for defence-in-depth.

No functionality was removed and no new dependency was introduced in reaching this state, which should make this change set straightforward to review, merge, and deploy behind the two environment variables specified in Section 10.

Prepared by	Reviewed by	Date
Application Security Engineering		29 August 2026